Q2E | Train Polynomial Regression

> E. Repeat everything from part D with polynomial regression and using SGD. Using validation loss, explore if your model overfits/underfits the data and report your findings. Ensure reproducibility by using a seed.

Part D, which this repeats:

> D. Train a Linear Regression model using the training data with four-fold cross-validation and with appropriate evaluation metric. Do this with a closed-form solution (using the Normal Equation or SVD) and with SGD. Then focus on SGD and perform Ridge, Lasso and Elastic Net regularization – try three values of penalty term and describe its impact. Explore the impact of other hyperparameters of your choice, such as learning rate (no need for grid search). Describe your findings. Select the best-performing regularization set of hyperparameters and train that model on the entire training dataset. For SGD, display the training and validation loss as a function of training epochs for the selected model. (Hint: In scikit-learn, max_iter counts epochs (full passes over the training set). To plot training/validation loss by epoch with SGDRegressor, set max_iter=1 and warm_start=True, then call fit(...) in a loop; after each call (each epoch), compute and record Train and Validation RMSE). Ensure reproducibility by using a seed.

IMPORTANT

This notebook uses only the train dataset (the first 584 days), so no modeling decision is influenced by the test data (data snooping). Every score below is a cross validation score on the training rows.

Answer Summary

Cross validation and metric

- Four time-ordered folds: each trains on every day before its 46-day validation block, scored by RMSE in rentals per day: it keeps the target's units and weighs large misses more than small ones.
- Each choice starts from the simpler option, which another replaces only if its mean validation RMSE is at least 1% lower.

Features

- Same fixed shapes as Q2D, from Q2B and Q2C on 9 attributes.

Polynomial regression: degree

- Only the continuous attributes are expanded. By closed form (SVD), degree 3 is carried: 4.16% below degree 2, the baseline, and a small gain over degree 1 (810.6 against 823.3).

Closed form vs SGD

- SGD at scikit-learn's defaults stops short of the closed form: 839.9 against 810.6.

Other hyperparameters

- `eta0`: only a narrow range works; 0.1 diverges, 0.001 is 32.45% worse, and the default 0.01 is kept.
- Schedule: `adaptive` replaces the default, 1.68% better than `invscaling` (825.8 against 839.9), by training longer.

Regularization

- A weak penalty ties no penalty: at alpha 0.001 all three land within 5 rentals of it.
- L1 bites first: at alpha 0.1 Lasso rises to 1295.1 against 933.8 for Ridge.
- A strong penalty underfits: at alpha 10 Lasso and Elastic Net predict the mean.

Selected model

- Elastic Net at alpha 0.001 (`adaptive`, `eta0` 0.01, degree 3), picked as required: 820.8 against 825.8 without a penalty.
- Trained on all 584 days and saved: training RMSE 600.9.

Loss by epoch

- The exact curve swings for about 17 epochs, then holds near 820 and never turns back up.
- The homework's loop never cuts `adaptive`'s step, so it never settles: its training RMSE ends above its validation RMSE (1011.9 against 881.9).

Overfitting and underfitting

- No clear sign of overfitting by degree: validation RMSE moves within a narrow band (810.6 to 845.8) while training RMSE falls steadily.
- The gap between training and validation RMSE is mostly the shift in time (Dummy shows a gap too). It is wider at higher degrees (239.1 at degree 1, 315.9 at degree 4), the only, mild, sign of overfitting.
- None of the penalties helps by 1% or more: the best, Elastic Net at alpha 0.001, is 5.0 below no penalty. More epochs do not overfit.

- Underfitting shows only with the larger penalties: from alpha 0.1 training and validation RMSE rise together, and at alpha 10 they approach Dummy's; degree 1 is not clearly worse than the higher degrees on validation.

Reproducibility

- Seed 42 for every SGD model; the folds are fixed by date and need none.

Imports

In [1]:
```
import joblib
import numpy as np
import pandas as pd
from IPython.display import display
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import SGDRegressor
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import TimeSeriesSplit
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import FunctionTransformer

import Q2.consts as Q2_CONSTS
from consts import TEST_SIZE, SEED, CARRY_MARGIN, CV_SPLITS, MAX_ITER, TOL,
POWER_T, LEARNING_RATE_SCHEDULES, ETA0S, ALPHAS, L1_RATIO, DEGREES

from utils import (
        read_csv_to_df,
        SPLIT,
        STATISTICS,
        PLOTTING,
        PIPELINES,
)

pd.set_option('display.max_rows', None)
pd.set_option('display.min_rows', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.float_format', '{:.1f}'.format)  # rentals to 1
decimal, like STATISTICS.results_df, in the tables it does not build
```

Fetch Data

In [2]:
```
df = read_csv_to_df(path=Q2_CONSTS.DATA_PATH, parse_dates=["dteday"])
df = df.rename(columns=Q2_CONSTS.SHORT_NAMES)
train_df, _ = SPLIT.chronological_split(df=df, date_col="Date",
test_size=TEST_SIZE)

ATTRIBUTES = ['Temp', 'FeelsLike', 'Humidity', 'Wind', 'Year', 'IsHoliday',
'IsWorkday', 'Season', 'Month', 'Weekday', 'Weather']
X, y = train_df[ATTRIBUTES], train_df[Q2_CONSTS.TARGET]
```

```
# left out: Casual + Registered = Rentals, so they would leak the target;
ID is a row label; Date is described by
# Year, Season, Month and Weekday
print(f"left out: {', '.join(column for column in train_df.columns if
column not in ATTRIBUTES + [Q2_CONSTS.TARGET])}")
```

```
left out: ID, Date, Casual, Registered
```

Setup

Folds

In [3]:

```
# Each fold trains on every day before its validation block of 46 days, the
way the model will predict the test days
# after the training days. The 4 blocks are the last 184 training days.
FOLDS = list(TimeSeriesSplit(n_splits=CV_SPLITS,
test_size=Q2_CONSTS.CV_TEST_SIZE).split(X))

pd.DataFrame({
        'training rows': [len(train) for train, _ in FOLDS],
        'first validation day': [f"
{train_df['Date'].iloc[validation[0]]:%Y-%m-%d}" for _, validation in
FOLDS],
        'last validation day': [f"
{train_df['Date'].iloc[validation[-1]]:%Y-%m-%d}" for _, validation in
FOLDS],
}, index=pd.RangeIndex(1, len(FOLDS) + 1, name='fold'))
```

Out[3]:

fold	training rows	first validation day	last validation day
1	400	2012-02-05	2012-03-21
2	446	2012-03-22	2012-05-06
3	492	2012-05-07	2012-06-21
4	538	2012-06-22	2012-08-06

Features

Attribute	Shape	Source
`FeelsLike`	cap at 0.573 (`'cap'`)	Q2B
`Temp`	dropped	Q2C: r 0.997 with `FeelsLike`
`Humidity`	hinge at 0.699, after clipping the one 0.0 day up to Q2B's lower IQR fence (0.172)	Q2B (hinge and clip); the hinge chosen over Q2B's floor during development, not shown
`Wind`	raw	Q2B
`Weather`	the code as a number	Q2B
`Year`	raw	Q2B keeps it in every model
`Season`	3 groups, {1} | {2, 4} | {3} (`'3 groups'`)	Q2B, fit with `Year`

Attribute	Shape	Source
`Month`	the code as a number	Q2B; the code chosen over its hinge during development, not shown
`Weekday`, `IsHoliday`	raw	chosen during development, not shown
`IsWorkday`	dropped	dropping it, with `Weekday` and `IsHoliday` kept, chosen during development, not shown

In [4]:
```
FITS = {attribute: STATISTICS.shape_fits(df=train_df, feature=attribute,
target=Q2_CONSTS.TARGET) for attribute in Q2_CONSTS.CONTINUOUS}
FITS['Season'] = STATISTICS.category_fits(df=train_df, feature='Season',
target=Q2_CONSTS.TARGET, control='Year')

def shape(*, attribute, name):
        """Q2B's shape of an attribute by name (ex 'cap'), with the setting
Q2B found, as a pipeline step."""
        return next(fit for fit in FITS[attribute] if fit.name ==
name).transformer

# Q2B: clip the one 0.0 day up to the lower IQR fence (0.172), then the
hinge
HUMIDITY_CLIPPED_HINGE = make_pipeline(FunctionTransformer(np.clip,
kw_args={'a_min': 0.172, 'a_max': None}), shape(attribute='Humidity',
name='hinge'))

SHAPES = {
        'Temp': 'drop',  # Q2C: r 0.997 with FeelsLike
        'FeelsLike': shape(attribute='FeelsLike', name='cap'),  # Q2B
        'Humidity': HUMIDITY_CLIPPED_HINGE,  # Q2B (hinge and clip); the
hinge chosen over Q2B's floor during development, not shown
        'Wind': 'passthrough',  # Q2B
        'Year': 'passthrough',  # Q2B keeps it in every model
        'IsHoliday': 'passthrough',  # chosen during development, not shown
        'IsWorkday': 'drop',  # dropping it, with Weekday and IsHoliday
kept, chosen during development, not shown
        'Season': shape(attribute='Season', name='3 groups'),  # Q2B, fit
with Year
        'Month': 'passthrough',  # the code as a number, Q2B; chosen over
its hinge during development, not shown
        'Weekday': 'passthrough',  # chosen during development, not shown
        'Weather': 'passthrough',  # the code as a number, Q2B
}
```

Runs

In [5]:
```
RUNS = {}  # every cross validation result, keyed by name
EXPAND = Q2_CONSTS.CONTINUOUS  # the attributes whose columns get
polynomial terms; Temp is dropped in SHAPES

def build(*, model, degree=1):
        """The pipeline of one experiment: SHAPES, standardized columns,
polynomial terms of the continuous columns above
        degree 1, model (see PIPELINES.build_pipeline)."""
```

```
        return PIPELINES.build_pipeline(model=model, attributes=ATTRIBUTES,
shapes=SHAPES, degree=degree, expand=EXPAND)

def score(*, name, pipeline):
        """Cross validate a pipeline and keep its result under name."""
        RUNS[name] = PIPELINES.cv_rmse(pipeline=pipeline, X=X, y=y,
folds=FOLDS)

def table(*, names, baseline=None):
        """One row per run in names: mean validation RMSE, mean training
RMSE, the most epochs for SGD, and the change from
        the baseline when one is given. A run more than ten times Dummy's
error shows as diverged (Method)."""
        return STATISTICS.results_df(
                runs={name: RUNS[name] for name in names},
                decimals=1,
                baseline=baseline,
                diverged_above=10 * RUNS['Dummy']['validation
rmse'].mean(),
        )

def lowest(*, names):
        """The run in names with the lowest mean validation RMSE."""
        return min(names, key=lambda name: RUNS[name]['validation
rmse'].mean())

def carried(*, baseline, names):
        """The decision rule: the run in names with the lowest mean
validation RMSE replaces the baseline (the simpler
        option) only if it is at least CARRY_MARGIN (1%) lower; otherwise
the baseline is kept."""
        best = lowest(names=names)
        gain = 1 - RUNS[best]['validation rmse'].mean() / RUNS[baseline]
['validation rmse'].mean()
        return best if gain >= CARRY_MARGIN else baseline
```

Method

- **Metric:** RMSE in rentals per day, because it is in the target's own units and weighs large misses more than small ones, and it is the score F reports on the test set. `validation rmse` is the mean over the 4 folds of the RMSE on each fold's validation days, days the model never saw; every choice is made on it. `train rmse` is the same on the days each model was fit on. A run whose mean validation RMSE is more than ten times Dummy's (predicting the training mean) is shown as `diverged`.
- **Folds:** four time-ordered folds (`TimeSeriesSplit`): each trains on every day before its 46-day validation block, so every fold trains on some 2012 days. Why 46 days: see Q2D's Method.
- **Rule:** each choice starts from a baseline, the simpler option: degree 2, and scikit-learn's default `eta0` (0.01) and schedule (`invscaling`). The option with the

lowest mean validation RMSE replaces the baseline only if it is at least 1% lower (`CARRY_MARGIN`); a smaller gain is not worth the added step. The `% vs baseline` column shows the change. The selected regularized model has no baseline: the lowest mean wins. Degree 1 is D's model, not polynomial regression: it is shown for reference but cannot be carried.

- **Pipeline:** shapes, standardized columns, polynomial terms, model. The target is standardized too, so `alpha` and `tol` mean the same thing in Q1 and Q2 (Q2D's Why the Target Is Standardized shows what this changes), and the predictions come back in rentals per day. Every step is fit on the training days of a fold only.
- **Polynomial terms:** at degree d, the standardized columns are expanded into every product of up to d of them (`PolynomialFeatures`, without a column of 1s) and standardized again. Only the continuous attributes are expanded: squaring a 0/1 column repeats it, and leaving the codes unexpanded was chosen during development, not shown.
- **Closed form:** least squares solved by SVD (`PIPELINES.SVDRegression`, numpy's `lstsq`). Some expanded columns are exact combinations of others: `LinearRegression` returns huge coefficients on them, while SVD returns the solution with the smallest coefficients.
- **Seed:** seed 42 for every SGD model. The folds need none: `TimeSeriesSplit` has no randomness, it always cuts the same blocks by date. Dummy and SVD are deterministic.
- **Optimism:** every choice is made on these folds, and Q2B's shape settings were found on all training days, so the scores are a little optimistic. The test set gives the honest estimate.

Baselines

Dummy predicts the training mean of each fold.

In [6]:
```
score(name='Dummy', pipeline=build(model=DummyRegressor()))
YMAX = 1.1 * RUNS['Dummy']['validation rmse'].mean()  # a run that diverges
is drawn at the top of a chart instead of flattening the rest

table(names=['Dummy'])
```

Out[6]:

	validation rmse	train rmse
Dummy	2521.6	1476.3

Notes / Observations

- **The reference every degree is read against:** predicting the training mean scores 2521.6 against 1476.3 on its training days: the shift in time opens a gap between training and validation RMSE for every model, Dummy included.

Degree

The closed form on the features at degrees 1 to 4. Degree 2 is the baseline (rule in Method); degree 1 is D's model, shown for reference only. Training and validation RMSE side by side show underfitting (both high and close) and overfitting (training RMSE falling while validation RMSE rises).

In [7]:
```
DEGREE_RUNS = {degree: f'Closed form, degree {degree}' for degree in
DEGREES}
for degree, name in DEGREE_RUNS.items():
        score(name=name, pipeline=build(model=PIPELINES.SVDRegression(),
degree=degree))

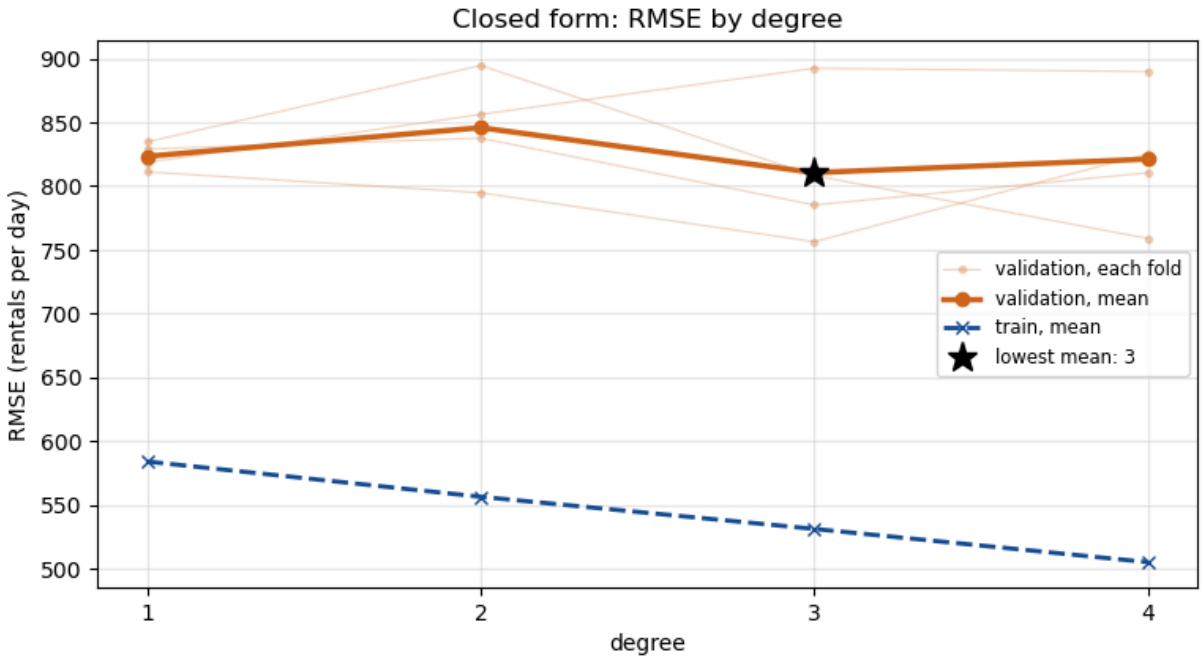
display(table(names=['Dummy', *DEGREE_RUNS.values()],
baseline=DEGREE_RUNS[2]))
PLOTTING.parameter_search(
        runs={degree: RUNS[name] for degree, name in DEGREE_RUNS.items()},
        parameter='degree',
        ymax=YMAX,
        title='Closed form: RMSE by degree',
        unit=Q2_CONSTS.TARGET_UNIT,
);

POLYNOMIAL = [DEGREE_RUNS[degree] for degree in DEGREES if degree > 1]  #
degree 1 is not polynomial regression
DEGREE = next(degree for degree, name in DEGREE_RUNS.items() if name ==
carried(baseline=DEGREE_RUNS[2], names=POLYNOMIAL))
pd.Series({'degree': DEGREE}, name='carried').to_frame()
```

	validation rmse	train rmse	% vs baseline
Dummy	2521.6	1476.3	+198.14%
Closed form, degree 1	823.3	584.2	-2.65%
Closed form, degree 2	845.8	556.5	baseline
Closed form, degree 3	810.6	531.4	-4.16%
Closed form, degree 4	821.2	505.3	-2.91%

Out[7]:

	carried
degree	3



Notes / Observations

- **Degree 3 is carried:** 4.16% below degree 2, the baseline, past the 1% the rule asks for.
- **No clear trend across degrees:** degree 2 is worse than degree 1 (845.8 against 823.3), and degree 3 gains little over degree 1 (810.6).
- **No clear sign of overfitting by degree:** training RMSE falls steadily (584.2 at degree 1 to 505.3 at degree 4) while validation RMSE moves within a narrow band.

SGD

`SGDRegressor` on the features at the carried degree, starting from scikit-learn's defaults with every setting written out: `penalty=None` (the default is a light L2 penalty), the learning rate schedule, `eta0`, `power_t`, `max_iter` and `tol`. `max_iter` is only a ceiling, raised above scikit-learn's default so that only `tol` ends a fit: once its training loss improves by less than `tol` for 5 epochs in a row.

In [8]:
```
CLOSED_FORM = DEGREE_RUNS[DEGREE]  # the carried closed form, the one SGD is compared against
SGD_SETTINGS = dict(penalty=None, learning_rate='invscaling', eta0=0.01, power_t=POWER_T, max_iter=MAX_ITER, tol=TOL, random_state=SEED)  # scikit-learn's defaults, written out (no penalty, a higher max_iter ceiling)
SETTINGS = {}  # the settings of every SGD run, keyed by name

def sgd(*, name, settings):
        """Cross validate SGD with these settings at the carried degree, and keep the settings under name."""
        SETTINGS[name] = settings
```

```
        score(name=name, pipeline=build(model=SGDRegressor(**settings),
degree=DEGREE))
```

SGD vs Closed Form

In [9]:
```
sgd(name='SGD', settings=SGD_SETTINGS)

table(names=[CLOSED_FORM, 'SGD'])
```

Out[9]:

	validation rmse	train rmse	most epochs
Closed form, degree 3	810.6	531.4	<NA>
SGD	839.9	544.9	14

Notes / Observations

- **At scikit-learn's defaults SGD stops short of the closed form:** 839.9 against 810.6, after at most 14 epochs.

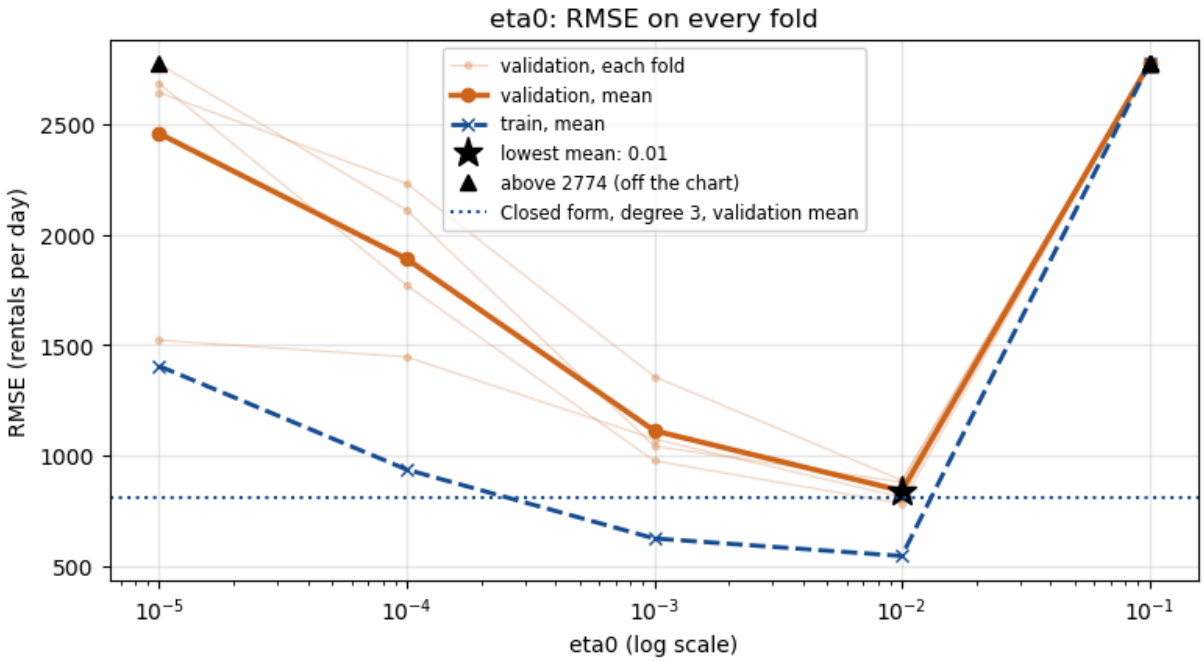
Learning Rate (eta0)

`eta0` is the starting step: under `invscaling` the step shrinks from it as training goes on. It is tried at 5 values, with the schedule at its default, and the default 0.01 is the baseline (rule in Method).

In [10]:
```
names = {eta0: f'eta0={eta0}' for eta0 in ETA0S}
for eta0, name in names.items():
        sgd(name=name, settings={**SGD_SETTINGS, 'eta0': eta0})
baseline = names[SGD_SETTINGS['eta0']]  # scikit-learn's default, 0.01

display(table(names=[CLOSED_FORM, *names.values()], baseline=baseline))
PLOTTING.parameter_search(
        runs={eta0: RUNS[name] for eta0, name in names.items()},
        parameter='eta0',
        log_x=True,
        reference=(CLOSED_FORM, RUNS[CLOSED_FORM]),
        ymax=YMAX,
        unit=Q2_CONSTS.TARGET_UNIT,
);
SGD_SETTINGS = {**SGD_SETTINGS, 'eta0': SETTINGS[carried(baseline=baseline,
names=names.values())]['eta0']}
```

	validation rmse	train rmse	most epochs	% vs baseline
Closed form, degree 3	810.6	531.4	<NA>	-3.50%
eta0=1e-05	2458.0	1405.0	40	+192.64%
eta0=0.0001	1890.1	936.3	76	+125.04%
eta0=0.001	1112.4	624.4	39	+32.45%
eta0=0.01	839.9	544.9	14	baseline
eta0=0.1	diverged	diverged	6	diverged



Notes / Observations

- **The default 0.01 is kept:** every other value is worse, and the closest, 0.001, is 32.45% higher.
- **Only a narrow range works:** 0.1 diverges, and 0.001 and below stop far short (1112.4 at 0.001).

Learning Rate Schedule

How the step changes from `eta0` as training goes on, at the carried `eta0`; `invscaling`, the default, is the baseline (rule in Method).

- `constant`: the step stays `eta0`.
- `optimal`: sets its own step from `alpha` and ignores `eta0`.
- `invscaling`: the step is `eta0 / t ** power_t`, shrinking as training goes on.
- `adaptive`: keeps `eta0` and divides it by 5 each time the training loss stalls for 5 epochs.

In [11]:
```
names = {schedule: f'learning_rate={schedule}' for schedule in
LEARNING_RATE_SCHEDULES}
for schedule, name in names.items():
        sgd(name=name, settings={**SGD_SETTINGS, 'learning_rate':
schedule})
baseline = names[SGD_SETTINGS['learning_rate']]  # scikit-learn's default,
invscaling

display(table(names=[CLOSED_FORM, *names.values()], baseline=baseline))
SGD_SETTINGS = {**SGD_SETTINGS, 'learning_rate':
SETTINGS[carried(baseline=baseline, names=names.values())]
['learning_rate']}
display(pd.Series(SGD_SETTINGS, name='carried').astype(str).to_frame())  #
as text: the 1 decimal float format would show power_t 0.25 as 0.2 and tol
0.001 as 0.0
```

	validation rmse	train rmse	most epochs	% vs baseline
Closed form, degree 3	810.6	531.4	<NA>	-3.50%
learning_rate=constant	1933.9	2605.6	9	+130.24%
learning_rate=optimal	diverged	diverged	113	diverged
learning_rate=invscaling	839.9	544.9	14	baseline
learning_rate=adaptive	825.8	538.5	48	-1.68%

	carried
penalty	None
learning_rate	adaptive
eta0	0.01
power_t	0.25
max_iter	10000
tol	0.001
random_state	42

Notes / Observations

- **`adaptive` replaces the default:** it is 1.68% below `invscaling` (825.8 against 839.9), past the 1% the rule asks for, by training longer (48 epochs against 14).
- **A step that stays at 0.01 does not settle:** `constant` ends with training RMSE above validation RMSE (2605.6); `optimal`, which sets its own step from `alpha` and ignores `eta0`, diverges.

Regularization

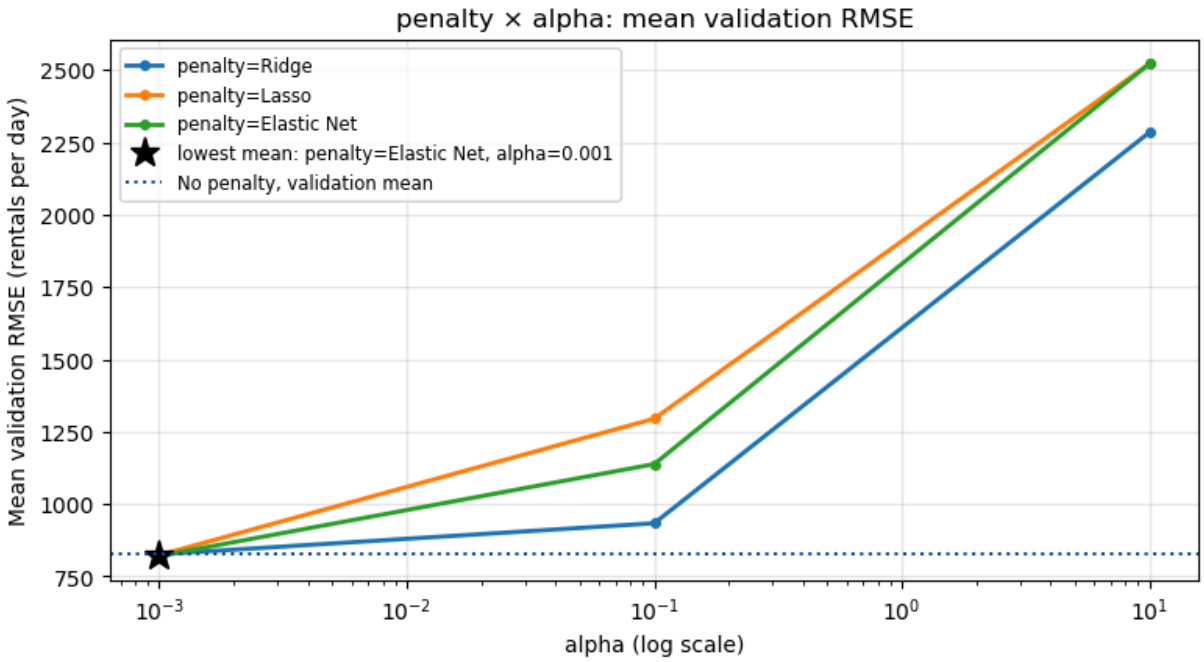
The carried SGD with a penalty: Ridge (`l2`), Lasso (`l1`) and Elastic Net (`elasticnet` , with `l1_ratio` fixed at 0.5, an even mix of both). `alpha` , the penalty's strength, is tried at 3 values a hundred times apart. Every run is compared with the same SGD without a penalty.

In [12]:
```
PENALTIES = {'Ridge': 'l2', 'Lasso': 'l1', 'Elastic Net': 'elasticnet'}
sgd(name='No penalty', settings=SGD_SETTINGS)

REGULARIZED = {(label, alpha): f'{label}, alpha={alpha}' for label in
PENALTIES for alpha in ALPHAS}
for (label, alpha), name in REGULARIZED.items():
        sgd(name=name, settings={**SGD_SETTINGS, 'penalty':
PENALTIES[label], 'alpha': alpha, **({'l1_ratio': L1_RATIO} if label ==
'Elastic Net' else {})})

display(table(names=['No penalty', *REGULARIZED.values()]))
PLOTTING.parameter_cross(
        runs={pair: RUNS[name] for pair, name in REGULARIZED.items()},
        lines='penalty',
        x='alpha',
        log_x=True,
        reference=('No penalty', RUNS['No penalty']),
        ymax=YMAX,
        unit=Q2_CONSTS.TARGET_UNIT,
);
```

	validation rmse	train rmse	most epochs
No penalty	825.8	538.5	48
Ridge, alpha=0.001	826.5	538.3	48
Ridge, alpha=0.1	933.8	562.0	48
Ridge, alpha=10	2284.0	1224.8	53
Lasso, alpha=0.001	823.7	534.3	54
Lasso, alpha=0.1	1295.1	718.3	48
Lasso, alpha=10	2520.9	1476.3	38
Elastic Net, alpha=0.001	820.8	534.9	50
Elastic Net, alpha=0.1	1138.2	644.4	46
Elastic Net, alpha=10	2520.9	1476.3	38



Notes / Observations

- **A weak penalty ties no penalty:** at alpha 0.001 all three land within 5 rentals of 825.8, Elastic Net lowest.
- **L1 bites first:** at alpha 0.1 Lasso rises to 1295.1 against 933.8 for Ridge; Elastic Net lands between them.
- **A strong penalty underfits:** at alpha 10 Ridge's training and validation RMSE rise together (1224.8 and 2284.0), and Lasso and Elastic Net predict the mean (2520.9).

Selected Model

The homework asks for one regularized model, so the 9 regularized runs are ranked by mean validation RMSE and the lowest is selected.

When none of the penalties improves on the model without one, the smallest `alpha` tends to win, because it changes the model the least. The selected model is then close to the model without a penalty: it is picked because the homework requires a regularized one, not because the penalty helps.

In [13]:
```
SELECTED = lowest(names=REGULARIZED.values())
display(pd.Series({**SETTINGS[SELECTED], 'degree': DEGREE},
name=SELECTED).astype(str).to_frame())  # as text: the 1 decimal float
format would show a small alpha as 0.0

table(names=['No penalty', SELECTED])
```

	Elastic Net, alpha=0.001
penalty	elasticnet
learning_rate	adaptive
eta0	0.01
power_t	0.25
max_iter	10000
tol	0.001
random_state	42
alpha	0.001
l1_ratio	0.5
degree	3

Out[13]:

	validation rmse	train rmse	most epochs
No penalty	825.8	538.5	48
Elastic Net, alpha=0.001	820.8	534.9	50

Notes / Observations

- **Elastic Net at alpha 0.001 is picked as required:** 820.8 against 825.8 without a penalty, the smallest alpha, as explained above.

Train on the Entire Training Set

The selected model refit once, with no folds, on all 584 training days, and saved. The saved pipeline takes the 11 raw attributes and builds its own columns, at the carried degree.

In [14]:
```
final = build(model=SGDRegressor(**SETTINGS[SELECTED]),
degree=DEGREE).fit(X, y)
Q2_CONSTS.MODEL_OUTPUT_DIR.mkdir(exist_ok=True)
joblib.dump(final, Q2_CONSTS.MODEL_OUTPUT_DIR /
'polynomial_regression.joblib')

pd.Series({
        'training rows': len(X),
        'epochs run': PIPELINES.final_model(pipeline=final).n_iter_,
        'train rmse': mean_squared_error(y, final.predict(X),
squared=False),
}, dtype=object).to_frame(SELECTED)  # object: keeps the counts whole
numbers
```

Out[14]:

	Elastic Net, alpha=0.001
training rows	584
epochs run	49
train rmse	600.9

Notes / Observations

- **Training RMSE on all 584 days is above the folds' (600.9 against 534.9), but the fit is sound:** it stops after 49 epochs, as on the folds, and it also trains on the last validation block, days no fold trains on.

Training and Validation Loss by Epoch

What the chart is for: following training and validation RMSE epoch by epoch, to see when training settles and whether more epochs start to overfit (validation RMSE turning up while training RMSE keeps falling). Each chart shows the selected model on every fold, for as many epochs as it ran on its longest fold.

Two ways to draw it:

- **The homework's loop:** a copy of the selected model with `max_iter=1` and `warm_start=True`, fit once per epoch on each fold's training days, recording training and validation RMSE after each `fit` call.
- **The exact curve:** epoch k is a fresh fit of the selected model with `max_iter=k` and the same seed, which is exactly the first k epochs of its real training. A fold's line goes flat once `tol` stops it.

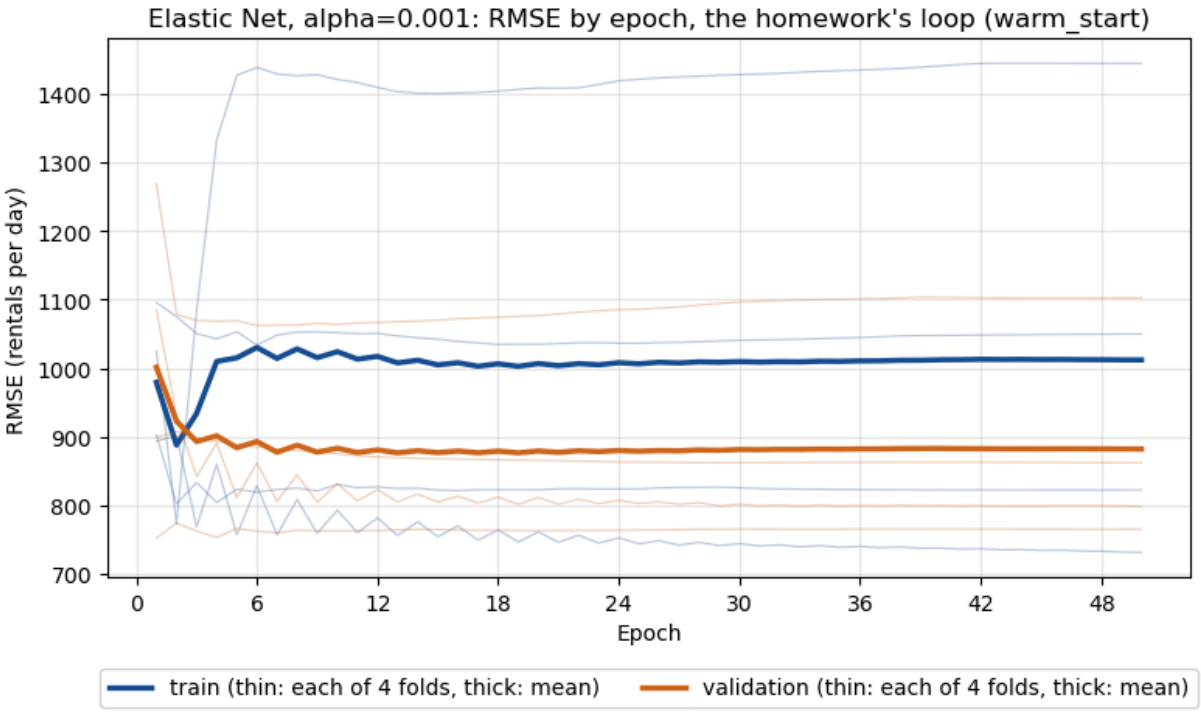
Why they differ: `warm_start=True` carries only the coefficients from one `fit` call to the next. Everything else starts over:

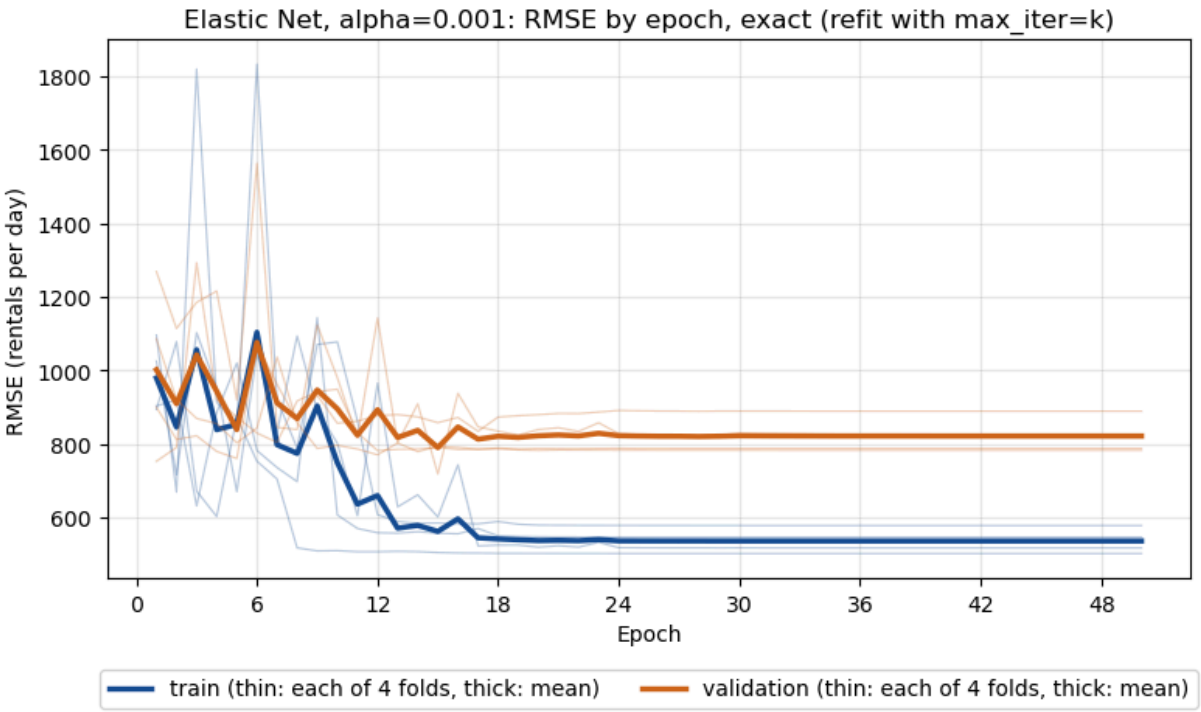
	One long fit (how the selected model trained)	The homework's loop (one `fit` call per epoch)
Coefficients	carry on	carry on (`warm_start=True`)
Step under `constant`	`eta0`	`eta0` , the same
Step under `invscaling` (`eta0 / t **power_t` , `t` counts the days seen)	keeps shrinking	starts again at `eta0` every epoch: `fit` sets `t` back to 1
Step under `adaptive` (divided by 5 after 5 epochs without improvement)	shrinks after each stall	stays at `eta0` : a 1-epoch fit can never stall 5 times
Order the days are visited in	a new shuffle every epoch	the same shuffle every epoch: each `fit` starts from the seed

So the loop replays the real training only under `constant`, and even then it visits the days in one fixed order. Under the other schedules it trains a different model: one whose step never gets smaller than where a single epoch leaves it. The exact curve shows how the selected model trained; the loop is drawn because the homework asks for it.

In [15]:
```
EPOCHS = int(RUNS[SELECTED]['epochs'].max())  # as many epochs as the
selected model ran on its longest fold
pipeline = build(model=SGDRegressor(**SETTINGS[SELECTED]), degree=DEGREE)
CURVES = {
        method: PIPELINES.epoch_rmse(pipeline=pipeline, X=X, y=y,
folds=FOLDS, epochs=EPOCHS, method=method)
```

```
        for method in ('warm_start', 'refit')
}
for method, label in (('warm_start', "the homework's loop (warm_start)"),
('refit', 'exact (refit with max_iter=k)')):
        PLOTTING.loss_curve(curve=CURVES[method], title=f'{SELECTED}: RMSE
by epoch, {label}', ymax=YMAX, unit=Q2_CONSTS.TARGET_UNIT);
```





Notes / Observations

- **The exact curve swings for about 17 epochs, then holds near 820:** `adaptive` starts at a step of 0.01 and cuts it each time the loss stalls; after the cuts both lines

go flat.

- **More epochs do not overfit:** validation RMSE never turns back up once the curve settles.

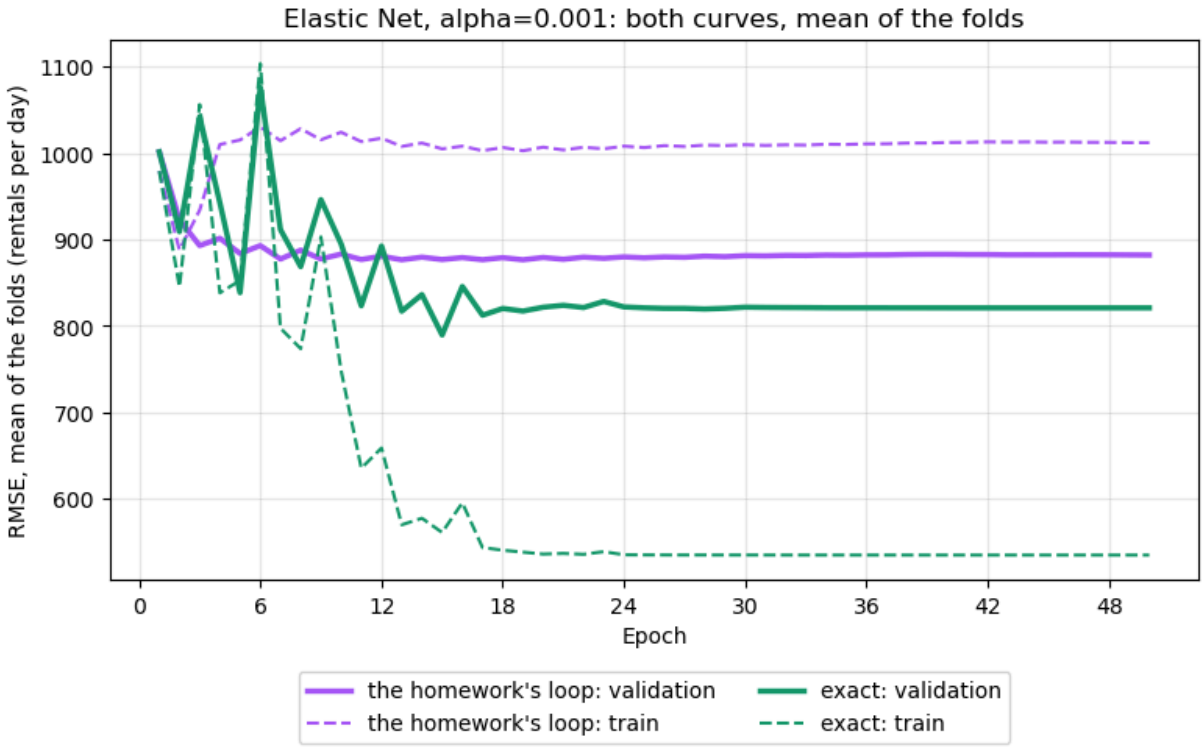
The Two Curves Side by Side

The mean of the folds for both curves on one chart, and the scores each ends on. The exact curve's last epoch is each fold's full fit, so it ends on the selected model's cross validation score from the Selected Model table; the homework's loop does not.

In [16]:
```
PLOTTING.loss_curve_comparison(curves=CURVES, title=f'{SELECTED}: both
curves, mean of the folds', ymax=YMAX, unit=Q2_CONSTS.TARGET_UNIT);
pd.DataFrame({
        'cross validation (Selected Model)': RUNS[SELECTED][['validation
rmse', 'train rmse']].mean(),
        "homework's loop, last epoch": CURVES['warm_start'].xs(EPOCHS,
level='epoch').mean(),
        'exact curve, last epoch': CURVES['refit'].xs(EPOCHS,
level='epoch').mean(),
}).T[['validation rmse', 'train rmse']]
```

Out[16]:

	validation rmse	train rmse
cross validation (Selected Model)	820.8	534.9
homework's loop, last epoch	881.9	1011.9
exact curve, last epoch	820.8	534.9



Notes / Observations

- **The loop never settles:** `adaptive` never cuts its step in a 1-epoch fit, so the loop keeps a step of 0.01 for all 50 epochs. Its training RMSE ends above its validation RMSE (1011.9 against 881.9), the mark of a step too large to settle.
- **The exact curve settles once its step is cut, and only it ends on the cross validation score (820.8):** here the loop shows a different, worse model, not how the selected model trained.

Overfitting and Underfitting

Validation loss read three ways: by degree (Degree), by penalty (Regularization) and by epoch (Training and Validation Loss by Epoch).

Notes / Observations

- **By degree:** no clear sign of overfitting: validation RMSE moves within a narrow band (810.6 to 845.8) while training RMSE falls steadily.
- **By penalty:** none of the penalties helps by 1% or more (the best, Elastic Net at alpha 0.001, is 5.0 below no penalty), and the larger ones raise training and validation RMSE together, so there is little overfitting for a penalty to cure.
- **By epoch:** the selected model's validation RMSE holds once it settles, so more epochs do not overfit.
- **The gap between training and validation RMSE is mostly the shift in time:** Dummy shows a gap too. It is wider at higher degrees (239.1 at degree 1, 315.9 at degree 4), the only, mild, sign of overfitting.
- **Underfitting shows only with the larger penalties:** from alpha 0.1 training and validation RMSE rise together, and at alpha 10 they approach Dummy's: Lasso and Elastic Net predict the mean (2520.9). Degree 1 is not clearly worse than the higher degrees on validation, so it does not clearly underfit either.